

Daybook Leave Management

Build plan for FastAPI backend, Streamlit frontend, SQLite database, and project Excel record storage

Current prototype	Streamlit prototype with employee, manager, and HR/admin views for leave booking, approvals, balances, wallchart, burnout, statistics, employees, and organisation overview.
Target build	Keep the same mock prototype UI style while moving business logic and persistence into FastAPI, SQLite, and a project Excel workbook for new entries and records.
Main stack	Backend: FastAPI. Frontend: Streamlit. Database: SQLite. Record workbook: data/daybook_records.xlsx using openpyxl or pandas.

1. Product Scope

- Employee users can sign in, view allowance, book leave, cancel pending leave, and see leave history and personal details.
- Managers can review direct-report requests, approve or reject with notes, inspect clashes, view wallchart, team statistics, and burnout signals.
- HR/admin can view organisation overview, all requests, employees, statistics, policy dates, and add new employees from the Employees screen.
- The HR Add Employee feature must visually follow the current mock prototype: same sidebar pattern, compact forms, metrics, tables, cards, and calm Daybook styling.
- All new employee records, leave requests, approvals, cancellations, and HR-maintained records must be stored in SQLite and appended to an Excel workbook in the project.

2. Target Architecture

Layer	Responsibility	Implementation
Streamlit FE	Render the prototype-style UI, collect inputs, show role-specific pages, and call API endpoints.	Split current app into pages/components plus frontend/api_client.py. Keep current Daybook theme and interaction pattern.
FastAPI BE	Own auth, role permissions, validation, leave workflow, employee creation, analytics, and Excel write coordination.	Routers for auth, employees, leave requests, approvals, balances, calendar, analytics, and records.
SQLite DB	Primary durable app store for users, employees, balances, leave requests, decisions, policy dates, and settings.	SQLAlchemy models backed by data/daybook.sqlite3 with seed script for demo data.
Excel records	Project-local workbook for newly added entries and records, useful for handoff, audit, and buildathon review.	data/daybook_records.xlsx with sheets for Employees, LeaveRequests, Decisions, Balances, PolicyDates, and ActivityEvents.

3. Suggested Folder Structure

```
backend/  
app/  
main.py  
database.py  
models.py  
schemas.py  
deps.py  
routers/  
auth.py  
employees.py  
leave_requests.py  
approvals.py  
balances.py  
calendar.py  
analytics.py  
records.py  
services/  
leave_policy.py
```

leave_workflow.py
 employee_service.py
 excel_records.py
 seed.py
 frontend/
 streamlit_app.py
 api_client.py
 pages/
 dashboard.py
 book_leave.py
 approvals.py
 wallchart.py
 employees.py
 data/
 daybook.sqlite3
 daybook_records.xlsx
 requirements.txt

4. Backend Tasks - FastAPI

ID	Task	Acceptance Criteria
BE-01	Create FastAPI scaffold, settings, router registration, CORS config, and GET /health.	Uvicorn starts locally and /health returns service status.
BE-02	Add SQLite via SQLAlchemy and seed current prototype data.	data/daybook.sqlite3 is created with demo employees, balances, requests, holidays, locked dates, and settings.
BE-03	Define ORM models and Pydantic schemas.	Models cover employees, roles, leave types, balances, leave requests, request decisions, policy dates, activity events, and settings.
BE-04	Implement prototype auth and role dependency helpers.	Streamlit can log in with demo emails and receive current user profile and role.
BE-05	Implement leave request create/list/detail/cancel endpoints.	Requests persist, validation errors are clear, and users only see permitted records.
BE-06	Implement manager and HR approval endpoints.	Only direct managers or enabled HR policy can approve/reject pending requests; decisions are audited.
BE-07	Move leave rules to services.	Working days, half-days, holidays, locked dates, balance checks, overlap warnings, and status transitions are unit tested.
BE-08	Implement HR Add Employee endpoints.	POST /employees validates required fields, unique email, manager link, role, leave balances, and stores the new employee.
BE-09	Implement Excel record writer service.	Every new employee, leave request, approval/rejection/cancellation, balance update, policy date, and activity event is appended to data/daybook_records.xlsx.
BE-10	Add dashboard, wallchart, employee, analytics, and admin read APIs.	Employee, manager, and HR pages receive role-scoped response data matching prototype behavior.

5. Database And Excel Storage Tasks

Store	Purpose	Implementation Detail
SQLite	Primary normalized application data.	Use data/daybook.sqlite3. Tables: employees, leave_types, leave_balances, leave_requests, request_decisions, policy_dates, activity_events, policy_settings.
Excel workbook	Project-local append-only workbook for all newly created entries and records.	Use data/daybook_records.xlsx. Create if missing. Keep separate sheets for Employees, LeaveRequests, Decisions, Balances, PolicyDates, ActivityEvents.
Write pattern	Keep app data reliable while preserving Excel records.	API writes to SQLite inside a transaction, then appends an Excel record with entity type, action, payload snapshot, actor, and timestamp.
Recovery	Avoid silent Excel failure.	If Excel write fails, log the error and return a clear admin-facing warning; keep SQLite as the source of truth.

6. HR Add Employee Feature Plan

Area	Build Detail
UI placement	Extend the existing HR Employees page with an Add Employee action and form section. Keep same Daybook sidebar, compact table, metrics, form controls, and neutral styling from the mock prototype.
Fields	Name, work email, role, title, department, manager, join date, phone, annual leave starting balance, earned leave starting balance, sick leave starting balance, and active status.
Validation	Required name/email/role/department, unique work email, valid manager for employee/manager roles, valid leave balances, and no future join date unless explicitly allowed.
Backend endpoint	POST /employees creates the employee, initializes balances, writes an activity event, and appends rows to the Excel Employees and Balances sheets.
Frontend behavior	On success, show the new employee in the Employees table immediately, refresh dashboard counts, and show a prototype-style success message.
Permissions	Only HR/admin can add employees. Managers and employees can view allowed profile data but cannot create employee records.

7. Frontend Tasks - Streamlit

ID	Task	Acceptance Criteria
FE-01	Create api_client.py for all FastAPI calls.	Streamlit pages call centralized helper functions instead of mock state.
FE-02	Replace in-memory data with API-backed state.	Refreshes preserve requests, approvals, and HR-created employees.
FE-03	Update login flow to call FastAPI.	Role-specific navigation still matches prototype after sign in.
FE-04	Wire Book Leave and My History to API.	Backend validation messages appear in Streamlit using the existing form style.
FE-05	Wire Approvals, Wallchart, Burnout, Statistics, Overview, All Requests, and Employees pages.	Current mock UI remains familiar while showing persisted data.
FE-06	Add HR Add Employee UI inside Employees page.	Form follows prototype styling, uses Streamlit controls, shows validation messages, and refreshes the employee table.
FE-07	Add loading, empty, and error states.	Backend outages and validation failures are clear without breaking layout.

8. API Endpoint Plan

Area	Endpoints
Auth	POST /auth/login, GET /auth/me, POST /auth/logout
Employees	GET /employees, POST /employees, GET /employees/{id}, PATCH /employees/{id}, GET /employees/{id}/balances
Leave requests	GET /leave-requests, POST /leave-requests, GET /leave-requests/{id}, POST /leave-requests/{id}/cancel
Approvals	GET /approvals/pending, POST /leave-requests/{id}/approve, POST /leave-requests/{id}/reject

Area	Endpoints
Calendar	GET /calendar/wallchart, GET /calendar/policy-dates, POST /calendar/policy-dates
Analytics	GET /analytics/dashboard, GET /analytics/team, GET /analytics/org, GET /analytics/burnout
Records	GET /records/excel/status, POST /records/excel/rebuild for HR/admin maintenance

9. Milestones

Phase	Scope	Done When
Phase 0 - Baseline	Document current prototype flows and capture seed data.	Employee, manager, and HR behaviors are mapped as expected behavior.
Phase 1 - Backend Foundation	FastAPI, SQLite, models, seed data, auth, profile, balances.	Backend starts independently and serves seeded data.
Phase 2 - Leave Workflow	Create/cancel leave, approve/reject, validation, activity audit.	Core leave lifecycle works through API with tests.
Phase 3 - HR Employee Management	Add employee endpoint, balance initialization, HR Streamlit form, Excel workbook append.	HR can add a user through the Employees page and the record appears in SQLite and data/daybook_records.xlsx.
Phase 4 - Streamlit Integration	Replace mock session data across all pages.	The UI keeps the prototype look and reads/writes via FastAPI.
Phase 5 - Analytics And Admin	Wallchart, stats, burnout, policy dates, all requests, records status.	Role-based reporting and admin screens match the prototype scope.
Phase 6 - QA And Packaging	Tests, README, run commands, seed/reset, and demo checklist.	A reviewer can install, seed, run API, run Streamlit, and demo the app.

10. Testing Plan

- Unit tests for leave_policy.py: working days, half-days, weekends, public holidays, locked dates, and invalid date ranges.
- Unit tests for leave_workflow.py: allowed status transitions, invalid transitions, approval/rejection notes, and actor permissions.
- API tests with FastAPI TestClient: login, create request, approve/reject, cancel, HR add employee, list employees, and role-scoped filtering.
- Excel tests using a temporary workbook: adding employee creates Employees and Balances rows; leave actions append LeaveRequests and Decisions rows.
- Streamlit smoke checklist: employee booking, manager approval, HR add employee, HR overview, wallchart, statistics, and sign out/sign in.

11. Local Run Plan

Step	Command / Action
Install	pip install -r requirements.txt
Seed	python -m backend.app.seed
Start API	uvicorn backend.app.main:app --reload --port 8000
Start FE	streamlit run frontend/streamlit_app.py
Storage files	SQLite: data/daybook.sqlite3. Excel: data/daybook_records.xlsx.

12. Definition Of Done

- All prototype flows are backed by FastAPI and persist to SQLite.
- All newly created entries and records are appended to data/daybook_records.xlsx inside the project.
- HR can add employees from a Streamlit screen that matches the current mock prototype UI.
- Role scoping is enforced in FastAPI, not only hidden in Streamlit navigation.
- Business rules have unit tests and return clear validation messages.
- README explains setup, seed, run commands, demo accounts, storage files, and known limitations.